

Спецификация и план разработки

Тема проекта: Визуализация алгоритма Флойда-Уоршелла

1. Общие положения

Проект представляет собой приложение с графическим интерфейсом для визуализации алгоритма нахождения кратчайших путей между всеми парами вершин во взвешенном ориентированном графе - алгоритма Флойда-Уоршелла.

Приложение позволит пользователю наглядно увидеть, как динамическое программирование последовательно улучшает оценки длин путей при добавлении промежуточных вершин.

2. Функциональные требования

Основные возможности пользователя

Добавление вершин: Создание новой вершины на холсте по двойному щелчку мыши. Новая вершина появляется в точке щелчка, ей автоматически присваивается уникальный идентификатор (буква или номер). Вершина сразу становится доступной для редактирования.

Редактирование ребер: Создание ориентированного ребра (дуги) между двумя вершинами осуществляется следующим образом: пользователь выбирает первую вершину (начало дуги) щелчком мыши, затем выбирает вторую вершину (конец дуги) щелчком (или через контекстное меню: правый клик на вершине → пункт "Добавить ребро из...", затем выбор конечной вершины). После выбора пары вершин появляется диалоговое окно для задания веса ребра (целое или вещественное число). Вес может быть положительным, отрицательным или нулевым.

Удаление элементов: Удаление выбранной вершины или ребра выполняется клавишей "Delete" (или через контекстное меню → пункт "Удалить"). При удалении вершины автоматически удаляются все инцидентные ей рёбра.

Перемещение вершин: Перетаскивание вершин по холсту с зажатой левой кнопкой мыши для удобного расположения графа.

Ввод/Вывод данных:

Ручной ввод: Задание матрицы смежности графа в табличном виде прямо в интерфейсе приложения (отдельная вкладка или диалоговое окно с редактируемой таблицей $n \times n$, где в ячейки вводятся числа или INF).

Загрузка из файла: Чтение графа из текстового файла (формат: первая строка - количество вершин n ; следующие n строк - координаты каждой вершины (X, Y) в виде двух целых чисел, разделённых пробелом; затем матрица смежности $n \times n$, где INF обозначает отсутствие ребра).

Сохранение в файл: Экспорт текущего состояния графа в тот же текстовый формат.

Управление визуализацией (Алгоритм):

Запуск: Кнопка "Запустить алгоритм" - выполняет алгоритм непрерывно до завершения с задержкой между шагами.

Пошаговый режим: Кнопки "Шаг вперед" / "Шаг назад" - выполняют переход на одну итерацию внутреннего цикла с обновлением отображения.

Индикация: На каждом шаге алгоритма отображаются три счётчика:

- **k** (внешний цикл) - текущая промежуточная вершина (выделяется на холсте, например, обводкой красного цвета и подсветкой метки);
- **i** (средний цикл) - вершина-источник (выделяется синим цветом);
- **j** (внутренний цикл) - вершина-назначение (выделяется зелёным цветом).

Все три счётчика отображаются в панели состояния и в логе выполнения.

Обновление ребер: При улучшении кратчайшего пути (условие $D[i][k] + D[k][j] < D[i][j]$) ребро (i, j) на холсте временно выделяется: меняет цвет на ярко-зелёный и увеличивает толщину на 2 пикселя. Выделение сохраняется до следующего шага, после чего ребро возвращается к обычному стилю. В матрице соответствующий элемент обновляется и подсвечивается жёлтым цветом. Если ребро отсутствовало (было INF), оно рисуется пунктирной линией после обновления.

Взаимодействие с результатом:

Матрица результатов: Отображение текущей и финальной матрицы кратчайших расстояний в таблице справа от холста. Значения, которые были обновлены на последнем шаге, подсвечиваются жёлтым фоном.

Просмотр пути: Возможность кликнуть на ячейку финальной матрицы (между вершинами i и j) и увидеть на холсте путь (цепочку ребер), соответствующий этому кратчайшему пути (восстановление по матрице next). Путь выделяется жирными рёбрами оранжевого цвета, вершины пути обводятся оранжевым.

Уточнение требований после сдачи версии 1:

Сделать перемещаемой границу между холстом и правой панелью, при перемещении ширина поля лога должна меняться так, чтобы оставаться максимально возможной.

В логе отмечать:

Начало новой итерации крупного цикла – пустой строкой и объявлением с красной строкой(отступом).

Начало новой итерации среднего цикла – красной строкой(отступом).

Уточнение требований после сдачи версии 2:

Переименовать кнопки: "Запустить" в "Выполнить", "Вперед" в "Вперёд".

При удалении вершины остальные не должны перемещаться.

3. Проектирование пользовательского интерфейса

Ниже представлен макет главного окна приложения.

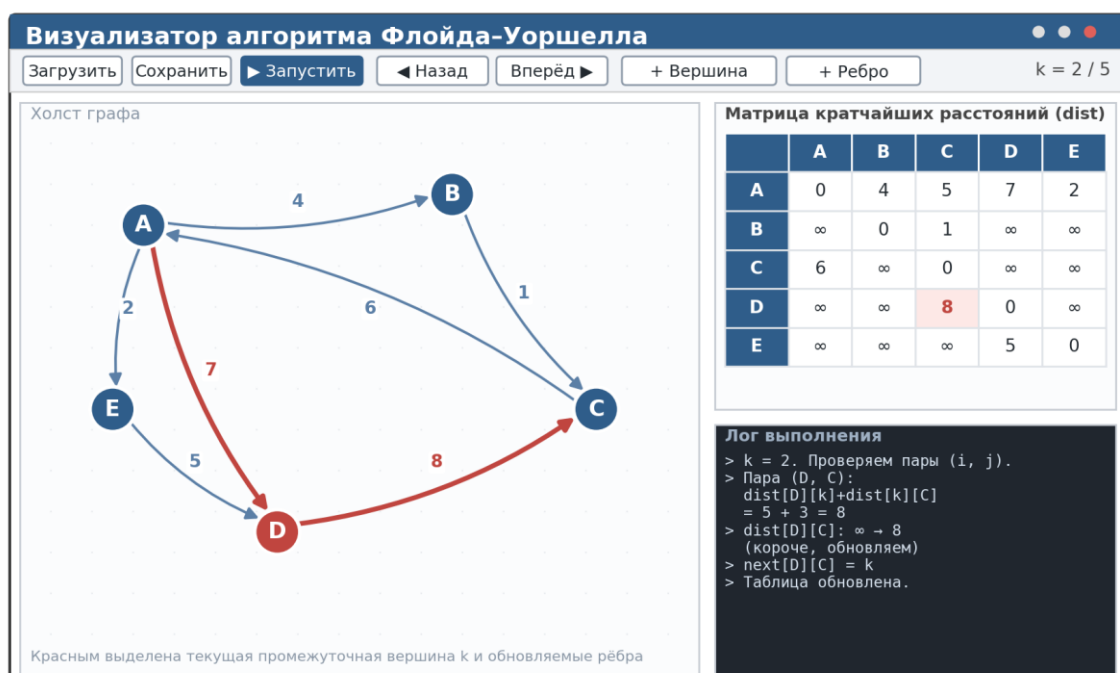


Рис. 1. Макет графического интерфейса.

Пояснение: Холст занимает левую часть окна. Правая панель разделена на область матрицы (таблица) и область лога/текстовых пояснений.

4. Процесс визуализации

При нажатии кнопки "Шаг вперед" или в режиме автозапуска выполняются следующие действия:

Текущие переменные циклов отображаются в панели состояния:
"Итерация $k = 2$, $i = 1$, $j = 3$ ".

Текущая промежуточная вершина k выделяется на холсте (обводится красным цветом). Вершина i выделяется синим, вершина j - зелёным.

В логе появляется сообщение:
"Проверяем $D[1][2] + D[2][3] = 3 + 1 = 4$? $D[1][3] = 5$ ".

Визуально пробегаются пары (i, j) - на каждом шаге соответствующие вершины подсвечиваются.

Если $D[i][k] + D[k][j] < D[i][j]$, то:

- на холсте ребро (i, j) меняет цвет на ярко-зелёный и увеличивается толщина;

- в логге: **"Найден более короткий путь из A в C: $3 + 1 = 4 < 5$ ";**
- в матрице значение обновляется, ячейка подсвечивается жёлтым.

Если неравенство не выполнено ($D[i][k] + D[k][j] \geq D[i][j]$), то:

- в логге выводится: **"Улучшение не требуется: $D[1][2] + D[2][3] = 4 \geq D[1][3] = 4$ "** (или аналогичное сообщение с конкретными значениями);
- ребро и ячейка матрицы остаются без изменений, никакая подсветка не применяется.

Обновление матрицы: Таблица справа перерисовывается с новыми значениями (изменённые ячейки подсвечиваются).

Окончание: Алгоритм завершается, когда $k == n$. В логге: **"Алгоритм завершен. Найдены все кратчайшие пути."** Все счётчики сбрасываются.

5. План разработки (Итерации)

Итерация 0: Спецификация и план разработки, прототип (Дедлайн: 10 июля)

[Общее]: Создать репозиторий.

[Гриценко К.В. (Модель/Логика)]: Создать необходимые классы. Реализовать алгоритм Флойда-Уоршелла на массивах (без GUI, с вызовом из main для теста).

[Кузьмин Г.И. (Интерфейс/View)]: Разработать макет окна на Swing. Сделать холст (JPanel + Graphics2D) для отрисовки графа (круги, стрелки).

[Харченко Я.К. (Контроллер/Ввод)]: Сделать кнопку "Загрузить". Реализовать кнопки "Шаг вперед", "Шаг назад" (стек состояний).

Результат: Запускаемое окно с пустым холстом, кнопками. Алгоритм работает, но результаты выводятся в консоль (или просто не шагово).

План тестирования.

1. Модульное тестирование алгоритма

Тестирование класса FloydWarshall должно включать проверку на графе без отрицательных циклов: загружается тестовый граф из 3-4 вершин с известными кратчайшими путями, запускается runFull(), после чего матрица dist сравнивается с эталонной матрицей, вычисленной вручную. Проверяется, что значения совпадают с точностью до $1e-9$. Для графа с отрицательными весами, но без отрицательных циклов, проверяется корректность обновлений.

Для обнаружения отрицательного цикла создаётся граф с циклом $A-B=1$, $B-C=1$, $C-A=-3$, запускается алгоритм, проверяется `hasNegativeCycle() == true`. Для графа из одной вершины проверяется, что матрица 1×1 содержит 0, алгоритм завершается без ошибок. Также тестируется восстановление путей: после выполнения алгоритма на графе без отрицательных циклов для каждой пары вершин восстанавливается путь и проверяется, что сумма весов рёбер пути равна `dist[i][j]` и что последовательность вершин корректна (первая вершина = i , последняя = j , каждая следующая достижима из предыдущей по матрице `next`). Проверяется поведение при запросе пути между вершинами, между которыми нет пути — возвращается пустой список. Проверяется пошаговый режим: после каждого вызова `step()` корректно обновляются счётчики `currentK`, `currentI`, `currentJ`, а `lastUpdates` содержит информацию о том, было ли улучшение на данном шаге, с правильными значениями `from`, `to`, `value`.

2. Модульное тестирование модели данных (Graph)

Тестирование класса `Graph` включает добавление вершины с уникальным ID - вершина появляется в списке, матрица перестраивается до размера $n \times n$. Добавление вершины с существующим ID возвращает `false` и не изменяет граф. Автогенерация ID проверяется последовательным добавлением вершин: первая получает ID "A", вторая "B" и т.д.; после 26 вершин проверяется генерация "V26". Добавление ребра между существующими вершинами корректно обновляет ячейку матрицы. Добавление ребра с несуществующей вершиной игнорируется. Удаление вершины проверяется: вершина удаляется из списка, все рёбра, инцидентные ей, становятся INF в матрице, остальные рёбра сохраняются, индексы остальных вершин пересчитываются корректно. Удаление ребра между существующими вершинами устанавливает значение в INF. Поиск индекса вершины по ID возвращает правильный индекс или -1 для несуществующей вершины. Получение матрицы смежности возвращает копию, изменение которой не влияет на внутреннее состояние графа. Перестроение матрицы после удаления вершины проверяется на сохранность данных: если были рёбра между оставшимися вершинами, они не теряются.

3. Интеграционное тестирование (алгоритм и модель)

После загрузки графа через `Graph` и инициализации `FloydWarshall` проверяется, что `dist` корректно копирует матрицу смежности, а `next` инициализируется правильно: для $i \neq j$, если есть ребро, `next[i][j]=j`, иначе -1. Проверяется, что вызов `runFull()` после `init()` корректно обновляет обе матрицы. Проверяется, что повторный вызов `init()` сбрасывает состояние

алгоритма. Проверяется восстановление пути: путь, полученный через `reconstructPath`, должен проходить через вершины, для которых `next[from][to]` не равен -1. Проверяется, что для недостижимых вершин `reconstructPath` возвращает пустой список.

4. UI-тестирование

Загрузка графа из файла: создаётся текстовый файл в правильном формате (`n`, координаты вершин, матрица смежности с INF), нажимается кнопка "Загрузить", выбирается файл, проверяется, что вершины отрисованы на холсте в указанных координатах, рёбра отрисованы с правильными весами, матрица смежности отображается в правой панели. Проверяется обработка некорректного файла - приложение не падает, в логе появляется сообщение об ошибке.

Добавление вершины: двойной клик на холсте создаёт вершину в точке клика с автоматическим ID, граф перерисовывается, матрица перестраивается с добавлением строки и столбца INF, в логе появляется сообщение о добавлении. Добавление вершины в пустой граф также работает корректно.

Редактирование ребра: выбирается первая вершина (начало), затем вторая (конец), появляется диалог ввода веса, после ввода числа ребро отображается на холсте, матрица обновляется. Проверяется ввод отрицательного веса, нулевого веса, вещественного числа (3.5). Проверяется отмена ввода - ребро не создаётся.

Удаление элементов: выбор вершины и нажатие Delete удаляет вершину и все инцидентные рёбра, холст и матрица обновляются. Выбор ребра (клик на линии между вершинами) и нажатие Delete удаляет ребро, в матрице устанавливается INF.

Перемещение вершин: перетаскивание вершины с зажатой левой кнопкой мыши перемещает вершину и все связанные рёбра перерисовываются с новыми координатами.

Пошаговое выполнение: после загрузки графа нажатие "Шаг вперёд" выполняет одну итерацию внутреннего цикла, в логе появляется сообщение вида "Проверяем $D[i][k] + D[k][j] = X + Y = Z$? $D[i][j] = W$ ", на холсте подсвечиваются вершины k (красным), i (синим), j (зелёным). При улучшении ребро (i, j) становится зелёным и утолщается на 2 пикселя, соответствующая ячейка матрицы подсвечивается жёлтым, в логе появляется сообщение об улучшении. При отсутствии улучшения в логе выводится сообщение

"Улучшение не требуется: $D[i][k] + D[k][j] = Z \geq D[i][j] = W$ ", подсветка не применяется. Нажатие "Шаг назад" возвращает состояние на один шаг: восстанавливаются матрицы `dist` и `next`, сбрасывается подсветка на холсте, матрица возвращается к предыдущему состоянию, лог обновляется.

Автозапуск: нажатие "Запустить" выполняет алгоритм непрерывно до завершения с задержкой между шагами (например, 500 мс), все шаги последовательно отображаются, по завершении в логе появляется сообщение "Алгоритм завершен. Найдены все кратчайшие пути.", все счётчики сбрасываются.

Матрица результатов: после завершения алгоритма в правой панели отображается финальная матрица кратчайших расстояний, все обновлённые на последнем шаге ячейки подсвечены жёлтым. При пошаговом выполнении матрица обновляется после каждого шага.

Просмотр пути: клик на ячейку матрицы (между вершинами i и j) подсвечивает на холсте соответствующий кратчайший путь: рёбра становятся жирными оранжевыми, вершины пути обводятся оранжевым. Если пути нет (INF), выводится сообщение в лог. Проверяется, что при клике на другую ячейку предыдущая подсветка сбрасывается.

5. Тестирование ввода/вывода

Сохранение графа в файл: после создания или загрузки графа нажимается "Сохранить", выбирается имя файла, проверяется, что файл создан и содержит корректные данные (n , координаты вершин, матрица смежности с INF). Round-trip тест: сохраняется текущий граф, затем загружается обратно, проверяется, что все вершины и рёбра сохранены корректно (количество вершин, веса рёбер, координаты совпадают с исходными). Проверяется сохранение в уже существующий файл - предложение перезаписать или выбрать другое имя.

Ручной ввод матрицы: открывается диалог/вкладка с таблицей $n \times n$, пользователь вводит числа или INF, после подтверждения граф перестраивается, вершины отрисовываются на холсте в круговом расположении.

6. Тестирование краевых случаев

Граф с 1 вершиной: добавление одной вершины, запуск алгоритма - матрица 1×1 содержит 0, алгоритм завершается корректно, восстановление пути из A в A возвращает список с одной вершиной. Добавление ребра в граф с 1 вершиной невозможно (нет второй вершины).

Граф с отрицательными весами без отрицательных циклов: создаётся граф $A-B=-5$, $A-C=10$, $B-C=2$, запускается алгоритм, проверяется, что $A-C$ становится -3 (через B), отрицательный цикл не обнаружен.

Граф с отрицательным циклом: создаётся цикл $A-B=1$, $B-C=1$, $C-A=-3$, запускается алгоритм, проверяется, что `hasNegativeCycle()` возвращает true, в логе появляется предупреждение. Проверяется, что при наличии отрицательного цикла восстановление пути не пытается построить бесконечный путь.

Большой граф (10-15 вершин): загружается полный граф со случайными весами, алгоритм выполняется без зависаний, все пути корректно восстанавливаются.

Пустой граф: запуск алгоритма без вершин - ошибка не возникает, выводится сообщение в лог, матрица пуста.

INF значения: проверяется, что при загрузке матрицы с INF соответствующие рёбра не создаются, при выводе матрицы INF отображается как " ∞ " или "INF".

7. Тестирование GUI-компонентов

Проверка масштабирования окна: при изменении размера окна холст и матрица корректно перерисовываются, пропорции сохраняются. Проверка, что минимальный размер окна не позволяет скрыть элементы управления. Проверка, что все кнопки имеют тултипы с пояснениями. Проверка, что при наведении на подсвеченное ребро или вершину появляется подсказка с текущим значением. Проверка цветовой схемы: контрастность цветов достаточная для восприятия, подсветка (красный, синий, зелёный, жёлтый, оранжевый) заметна на фоне. Проверка лога: при большом количестве сообщений появляется скролл, лог не теряет старые сообщения, каретка автоматически прокручивается к последнему сообщению.

8. Тестирование сборки и запуска

Сборка JAR через Maven: выполняется `mvn clean package`, проверяется, что создаётся исполнимый JAR-файл. Запуск на чистой машине с

установленной Java 21: `java -jar FloydWarshall.jar` — приложение запускается без ошибок, все функции работают. Проверка, что JAR не содержит внешних зависимостей (кроме стандартной библиотеки Java). Проверка, что при отсутствии Java 21 выводится понятное сообщение об ошибке.

Итерация 1: Базовая функциональность (Дедлайн: 13 июля)

[Гриценко К.В.]: Интегрировать алгоритм с визуализацией. Сделать метод `step()`, который выполняет одну итерацию k и возвращает список изменений (какие ребра обновились).

[Кузьмин Г.И.]: Реализовать динамическую отрисовку: отображение весов ребер, подсветку текущей вершины k , анимацию изменения цвета ребер при обновлении `dist`.

[Харченко Я.К.]: Реализовать загрузку графа из файла и парсинг в модель. Добавить таблицу для отображения матрицы `dist`.

Результат: Приложение позволяет загрузить граф, пошагово пройти алгоритм, видя изменения на холсте и в таблице.

Итерация 2: Улучшения (Дедлайн: 15 июля)

[Гриценко К.В.]: Реализовать восстановление пути по матрице `next`. Написать функцию, возвращающую список вершин для пути $i \rightarrow j$.

[Кузьмин Г.И.]: Реализовать "Drag-and-Drop" для вершин. Добавить контекстное меню (правый клик) для добавления/удаления ребер.

[Харченко Я.К.]: Связать клик по ячейке в матрице с отображением пути на холсте. Реализовать "Сохранение графа в файл".

Результат: Полноценное интерактивное приложение. Можно рисовать граф мышкой, запускать алгоритм и просматривать конкретные кратчайшие пути.

Итерация 3: Полировка и тесты (Дедлайн: 17 июля)

[Все]: Проверка на краевые случаи: граф с 1 вершиной, отрицательные веса (проверка на циклы).

[Гриценко К.В.]: Написать unit-тесты для алгоритма (JUnit 5): корректность на графах без циклов, с отрицательными весами без отрицательных циклов, определение отрицательного цикла, граф из 1 вершины.

[Кузьмин Г.И.]: Улучшить UI (иконки, тултипы, цветовая схема). Добавить полосу прогресса.

[Харченко Я.К.]: Собрать финальный .jar файл через Maven (запуск `java -jar`, совместимо с Java 21, без внешних зависимостей). Проверить запуск вне IDE на чистой машине. Написать разделы отчёта по своей части.

Результат: Готовый продукт, защищенный от ошибок, с красивым интерфейсом.

Роли:

Backend Developer (Гриценко К.В.):

Ответственность: Архитектура модели (Model), логика алгоритма, работа с файлами (ввод/вывод матриц).

UI/UX Developer (Кузьмин Г.И.):

Ответственность: Все, что касается графического интерфейса (Swing, Graphics2D), отрисовка графа, анимация, дизайн панелей.

Frontend/Controller Developer(Харченко Я.К.):

Ответственность: Связь между моделью и представлением (MVC), обработка событий мыши/клавиатуры, логика кнопок, управление состоянием (шаги), сборка проекта.